[10] - Data Science

Tasks

Analyze Data with R

Calculating Population Change Over Time with R

You work at the UN in urban planning and are interested in tracking population growth across major metropolitan regions. You are hoping that by looking at historical population numbers that you can predict future growth and help your team make decisions about resourcing.

Use what you've learned about the basics of R to calculate the population growth of Istanbul and create a short report.

If you get stuck or want to see an experienced developer tackle this project, click "get help" for a walkthrough video.

Note: Technically, we should use a geometric mean to calculate population growth, but we're using an arithmetic mean here to practice specific operators in R.

Tasks

4/10 complete
Mark the tasks as complete by checking them off

Creating Variables

1.
Istanbul is the largest city in Turkey and the fifth largest city in the world. It has experienced enormous growth over the past 50 years and is one of the world's 10 fastest growing metropolitan areas.
While the program that we will write can be used with data from any city, we'll start by using data from Istanbul and saving our data to variables. Using variables will allow us to swap out the data in the future.
The following chart is an abbreviated list of the population size by year in Istanbul. Take a moment to read over the data — you will need to refer back to this chart as you complete certain tasks.

Year	Population
1927	691000
1950	983000
2000	8831800

2017 15029231

2. First, create the variable `city_name` and set it equal to `"Istanbul, Turkey"` .
3. The dataset starts with the population value for the year 1927 and ends with 2017.
Create the variable `pop_year_one`. In the chart, find the population value for 1927 and set it equal to the variable `pop_year_one`.
4. Next, create the variable `pop_year_two`. Find the population for 2017 and set its value equal to the variable `pop_year_two`.

Using Variables to Perform Calculations

5. Using the variables that we just created, we're going to write a script that allows us to calculate the *annual percentage growth rate*. The annual percentage growth rate is the amount in which the population changes each year during a certain period.
First, create the variable `pop_change`. Calculate the difference in population between 2017 and 1927 and save the result to the variable `pop_change`. Feel free to print any of these variables if you want to check their values!
6. Before we calculate the annual percentage growth rate, we need to calculate the *percentage growth rate*. This is the percentage with which a population changes, but doesn't account for period of time during which the change takes place.
We can calculate percentage growth rate using the following formula:

```
percentage_gr <- (((pop_present - pop_past)/pop_past) * 100)
```

Copy to Clipboard
Create the variable `percentage_gr`.
Using the variable `pop_change`, calculate the annual percentage growth rate between 1927 and 2017 and assign the result to the variable `percentage_gr`.
7. Now that we have the percentage growth rate, we can calculate the annual percentage growth. Create a variable for `annual_gr`.
To calculate the annual percentage growth, take the result of the variable `percentage_gr` and divide it by the number of years elapsed. Set the result equal to the variable `annual_gr`.
8. Print the `annual_gr` by using the `print()` function.
9. Try using the same formula but changing the values for the years. You could pick a ten year period of your liking and see how population change between the earliest year in the decade and the last. This is the beauty of code and reproducibility, you can change the value of your variables and compute the same equation.

Call the function with correct arguments

10.
You've coded the calculation from scratch! At the top of your notebook, we've included a function named `calculate_annual_growth` that prints a sentence explaining the change in population. With your new knowledge of calling functions, and your understanding of variables and arguments, call the function to print a summary.
The `calculate_annual_growth` function takes five arguments:

- `year_one`
- `year_two`
- `pop_y1`
- `pop_y2`
- `city`

Pass in the correct values for each one- remember you already turned a few of them into variables! The others you can pass as values. Note: The argument `city` just corresponds to the city name as a string.
See the summary result that is printed as the result of the call!

```
---
title: "Introduction to R Syntax"
output: html_notebook
---
```{r error=TRUE}
```
```

```
calculate_annual_growth <- function(year_one,year_two,pop_y1, pop_y2,city) {
  annual_growth <- (((pop_y2 - pop_y1) / pop_y1) * 100) / (year_two-year_one)
  message <- paste("From", year one, "to", year two, "the population of", city, "grew by
approximately", annual_growth, "% each year.")
  print(message)
  return(annual growth)
}
# Write your code starting here:
# 1
city name <- c("Instabul", "Rurkey")
print(city_name)

# 2
pop_year_one <- 691000
print(pop_year_one)

# 3
pop year two <- 15029231
print(pop_year_two)

# 4
pop_change <- pop_year_two - pop_year_one
print(pop_change)

# 6
percentage_gr <- ((pop_change/pop_year_one) * 100)
print(percentage_gr)

# 7
annual_gr <- percentage_gr / pop_change

# 8
print(annual gr)


```

Day of the Race

It's the day of a citywide track race and you're helping out your friends! You'll be adding their jersey and color information to a list, helping display their information, and building a useful function to figure out what place your friends came in the race.

As you work through the code, make sure the syntax is correct, otherwise the results of the code will not update on the notebook preview. Use all resources so far to review if you need to, including this this very handy cheatsheet!

We've included a file named `solution.Rmd` for you to look at in case you get stuck. Feel free to copy that solution code into `notebook.Rmd` and save the code to see the final results.

Tasks

0/12 complete
Mark the tasks as complete by checking them off

Friends

1.
Your friends' names are Megan, Janet, and Tina. Create a vector named `friends` with their names as strings.
2.
You want to help your friends upload some personal information. The race accepts `jersey`, and `color` associated with a name. Add on to the existing `info_list` with the following information so that selecting the person's name from the list returns their jersey and color information:

Name	Jersey #	Color
Megan	1363	green
Janet	6729	green
Tina	7501	orange

3.
See the existing function `print_information`. It takes a string like `"Megan"` and prints out their information in a sentence. Call it to print Megan, Janet, and Tina's information.
4.
Bonus: instead of writing out `print_information()` three times, use a loop to iterate through `friends` to print information on each friend.

find_place()

5.
`race_results` contains the name of the race participants in the order that they finished. We want to write a function called `find_place()` that takes a runner's name and returns the position that they finished the race in. For example, `find_place("Francesca")` should return `2`.
In this step, start by declaring the function `find_place` and a parameter `runner`.
6.

Inside the function, start a for loop that iterates through the indices of `race_results`. Name your loop variable `place`. Note that the indices go from `1` to the length of `race_results`, and the variable `place` will increment by 1 every iteration of the loop.

7.

In the loop body, write an `if` statement that checks whether the `race_results` at the index `place` is equal to the parameter `runner`. If so, the function should return `place`. This stops the loop (since we no longer need to keep looking)

8.

The code after the loop will only run if the loop completes without the `runner`'s name being equal to anything in `race_results`. In this case, we should just return `0`.

For the last line of the function, return `0`.

9.

Call the `find_place` function on "Francesca" or any other runner's name in the vector at the top of the code. This is to check correctness! Verify that if `find_place` is given the argument `"Francesca"`, the result returned is `2`.

10.

What happens if you call the function with a name that doesn't exist in `race_results`? If none of the names match the name given to the function, the code would execute that line `return 0`.

Edit the function so that the function returns the position after last place for an unknown name instead.

11.

Apply the `find_place` function on the `friends` vector using `lapply()`.

12.

Now apply the same function on the `friends` vector using `sapply()`. Doesn't that look much better? Congratulations on completing this project!

````
---
title: "Day of the Race"
output: html_notebook
---

```{r}
# create friends vector here:
friends <- c("Megan", "Janet", "Tina")
```

```{r}
# add on to the list here:
info_list <- list(
  Esther = list(
    jersey = 3432,
    color = "purple"
  ),
  Feng = list(
    jersey = 4221,
    color = "blue"
  ),
  Megan = list(
    jersey = 1363,
    color = "green"
  ),
  Janet = list(
    jersey = 6729,
    color = "green"
  ),
  Tina = list(
    jersey = 7501,
    color = "orange"
  )
)
```

```{r}
print_information <- function(name) {
````
````

```
  print(paste(name, "is #", info_list[[name]]$jersey, "wearing the color",
info_list[[name]]$color))
}
# call the print_information function on the friends vector:
print_information("Megan")
print_information("Tina")
print_information("Janet")
```

```{r}
race_results <- c("Gi", "Francesca", "Lea", "Vivian", "Jessica", "Esther", "Mary",
"Yasmina", "Megan", "Janet", "Tiffany", "Kishan", "Feng", "Z", "Tina")
```

```{r}
# write find_place() here:
find_place <- function(runner) {
   for (place in 1:length(race_results)) {
      if (race_results[place] == runner) {
        return(place)
      }
   }
   return(length(race_results) + 1)
}
```


```{r}
# call and apply find_place() here:
find_place("Francesca")
find_place("Owen")
lapply(friends, find_place)
sapply(friends, find_place)
```

Analyze Data with R

Central Tendency for Housing Data in R

In this project, you will find the mean, median, and mode cost of one-bedroom apartments in three of the five New York City boroughs: Brooklyn, Manhattan, and Queens.

Using your findings, you will make conclusions about the cost of living in each of the boroughs. We will also discuss an important assumption that we make when we point out differences between the boroughs.

We worked with Streeteasy.com to collect this data. While we will only focus on the cost of one-bedroom apartments, the dataset includes a lot more information if you're interested in asking your own questions about the Brooklyn, Manhattan, and Queens housing market.

Tasks

0/13 complete
Mark the tasks as complete by checking them off

Observing your Data

1. We've imported data about one-bedroom apartments in three of New York City's boroughs: Brooklyn, Manhattan, and Queens. We saved the values to:

```
brooklyn_one_bed
manhattan_one_bed
queens_one_bed
```

In this project, we only care about the price of apartments, so we saved the price of apartments in each borough to:

```
brooklyn_price
manhattan_price
queens_price
```

If you want to see what the data stored in these variables looks like, you can type the variable names in a code block. When you click run, the variables (and their contents) will appear in rendered notebook on the right.

Find the Mean

2. Find the average value of one-bedroom apartments in Brooklyn and save the value to `brooklyn_mean`.

3. Find the average value of one-bedroom apartments in Manhattan and save the value to `manhattan_mean`.

4. Find the average value of one-bedroom apartments in Queens and save the value to `queens_mean`.

Find the Median

5. Find the median value of one-bedroom apartments in Brooklyn and save the value to `brooklyn_median`.

6. Find the median value of one-bedroom apartments in Manhattan and save the value to `manhattan_median`.

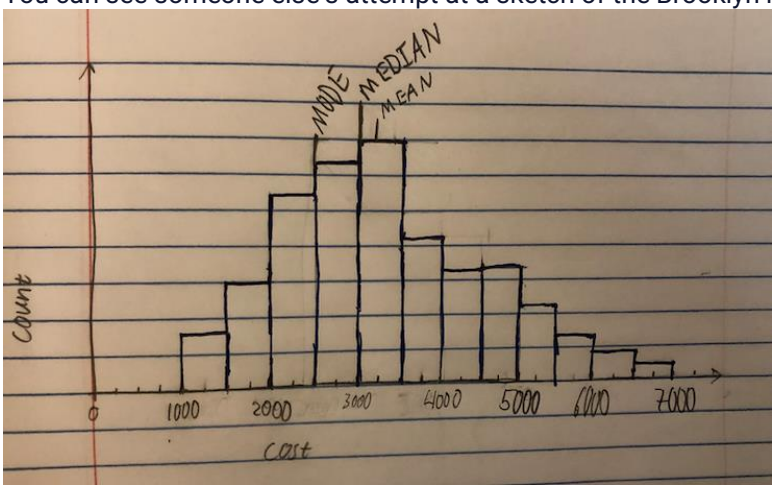
7. Find the median value of one-bedroom apartments in Queens and save the value to `queens_median`.

Find the Mode

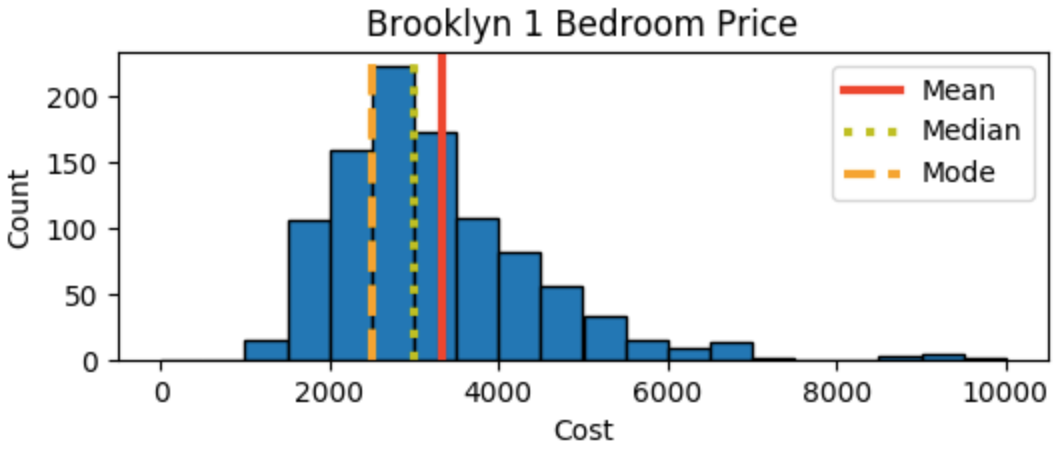
8. Find the mode value of one-bedroom apartments in Brooklyn and save the value to `brooklyn_mode`.
9. Find the mode value of one-bedroom apartments in Manhattan and save the value to `manhattan_mode`.
10 Find the mode value of one-bedroom apartments in Queens and save the value to `queens_mode`.

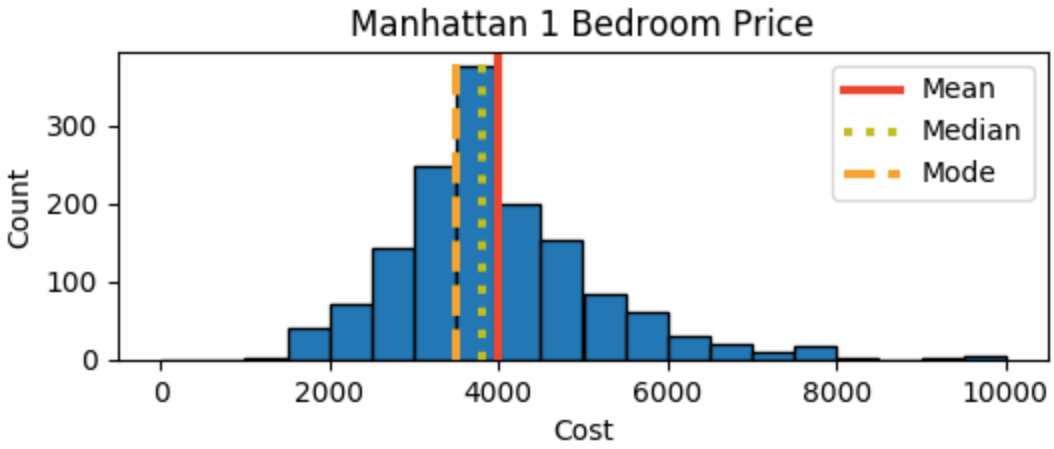
What does our data tell us?

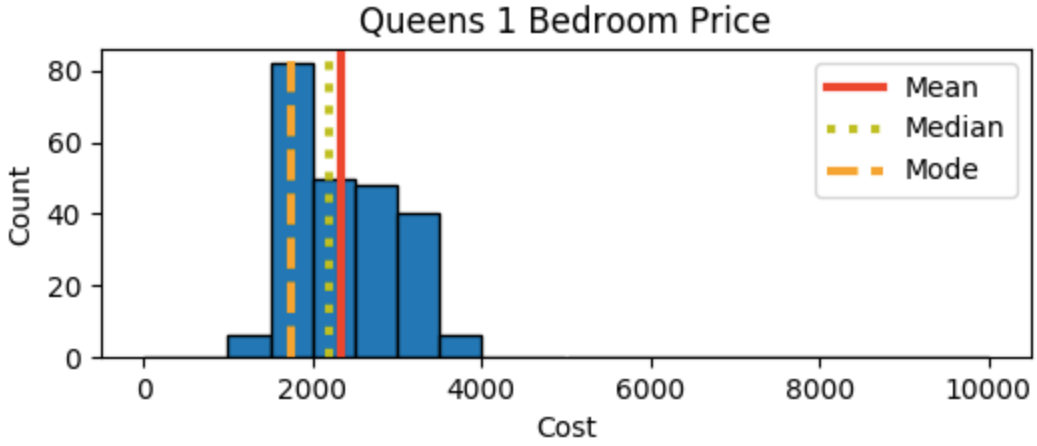
11.
Now what?
We don't find the mean, median, and mode of a dataset for the sake of it.
The point is to make inferences from our data. What can you say about the housing prices in Brooklyn, Queens, and Manhattan? Besides, "It's really expensive to live in any of them."
Take a minute to think through it. We added our thoughts to the hint.
12.
Did you make any assumptions when you drew inferences in the previous task?
If so, what assumptions did you make? We added our thoughts to the hint.
13.
Finally, think about what the histogram for each dataset will look like.
If you have the time, take a minute to make a rough sketch of the histograms for the cost of a one-bedroom apartment in Brooklyn, Manhattan, and Queens.
You can see someone else's attempt at a sketch of the Brooklyn histogram.



When you're finished, open the hint to take a look at the actual histograms for Brooklyn, Manhattan, and Queens.


Brooklyn 1 Bedroom Price
Mean
Median
Mode
Count
Cost
0
50
100
150
200
0
2000
4000
6000
8000
10000


Manhattan 1 Bedroom Price
Mean
Median
Mode
Count
Cost
0
100
200
300
0
2000
4000
6000
8000
10000


Queens 1 Bedroom Price
Mean
Median
Mode
Count
Cost
0
20
40
60
80
0
2000
4000
6000
8000
10000

Variance in Weather in R

https://www.codecademy.com/paths/analyze-data-with-r/tracks/statistics-in-r-skill-path/modules/r-stats-variance-and-standard-deviation/projects/variance-in-weather-r

You're planning a trip to London and want to get a sense of the best time of the year to visit. Luckily, you got your hands on a dataset from 2015 that contains over 39,000 data points about weather conditions in London. Surely, with this much information, you can discover something useful about when to make your trip!

In this project, the data is stored in a DataFrame. If you've never used a DataFrame before, we'll walk you through how to filter and manipulate this data. If you want to learn more about DataFrames, check out our R course.

Tasks

0/13 complete
Mark the tasks as complete by checking them off

Explore the Data

1.
All of the weather data is stored in a variable named `london_data`.
Print the first few rows of the dataset by calling `head(london_data)`.
Take a look at the browser to see the columns of this dataset. Here are two questions to ask yourself:
- How often were measurements taken?
- Which columns might be the most useful when thinking about planning a trip.

Comment out these print statements after looking through the dataframe `london_data`.

2.
Let's also take a look at how many rows we have. Print `nrow(london_data)`

Looking At Temperature

3.
Now that we've seen what the data looks like, let's dive into one of the more promising columns — `TemperatureC`. This column stores the temperature in Celsius.
To get a single column from a DataFrame, you can use this syntax:

```
one_column <- london_data$column_name
```

Copy to Clipboard
Create a variable named `temp` and set it equal to the `TemperatureC` column of `london_data`.

4.
We can now calculate descriptive statistics about this column. To begin, find the average temperature in London in 2015. Store it in a variable named `average_temp`.

5.
Calculate the variance of the temperature column and store the results in the variable `temperature_var`. Print the results.

6.
Calculate the standard deviation of the temperature column and store a variable named `temperature_standard_deviation`. Print this variable.
How would the variance and standard deviation help you plan a trip?

Filtering By Month

7.
The statistics we just calculated aren't very helpful when trying to plan a vacation since they describe the weather throughout an entire year.
If we could find a way to use the rows from only a certain month, that might help us find the best month to plan our trip.

Once again, print `head(london_data)` to see the first few columns of our DataFrame. Which column will help us get only the data points from January? In the browser you can scroll to the right to see more columns.

8.

We want to filter by the `"month"` column! The following line of code will create a variable that returns the data from the rows where `"month"` is `6`. These will be all of the rows from the month of June.

```
june <- london_data %>%
  filter(month == "06")
```

Copy to Clipboard

Create this variable for June.

9.Create a variable named `july` that contains all of the data points from July. The code to do this should look very similar to your code that created the June variable. This time, we're interested in month `"07"`.

10.

Calculate and print the mean temperature (`TemperatureC`) in London for both June and July using the `mean()` function.

What do these numbers tell you? If you wanted to visit London on the month that was, on average, cooler, which month would you pick? Look at the hint to see our thoughts!

11. Calculate and print the standard deviation of temperature in London for both June and July. Remember, the function you should use is `sd()`.

What do these numbers tell you? How might the standard deviation change your decision on when to visit London? Click on the hint to see our thoughts.

12. If you want to quickly see the mean and standard deviation of every month, use this block of code.

```
# Analyze by month
monthly_stats <- london_data %>%
    group_by(month) %>%
    summarize(mean = mean(TemperatureC),
              standard_deviation = sd(TemperatureC))
```

Copy to Clipboard

During which month would you most like to visit? If you wanted to pick the month with the least variable temperature, which one would you pick?

Explore on Your Own

13.

By looking at the mean and standard deviation of the temperature in London during each month of the year, we can get a sense of the best time to visit.

Looking at the spread of the data is an important statistic to consider if you are particularly sensitive to extreme days. For example, if you pick a month with a large standard deviation, you might have one day that is relatively cold while the following day is very hot.

Take some time to see if you can find more insights in this dataset. Here are some ideas we have for you:

- Look at columns other than `"TemperatureC"`. Can you find something interesting about the humidity or the air pressure? Can you find the rainiest month? London is notoriously rainy!
- Filter based on`"hour"`. Similar to how you filtered based on the month, are there certain hours that have higher variance than others?

Life Expectancy By Country

Over the course of the past few centuries, technological and medical advancements have helped increase the life expectancy of humans. However, as of now, the average life expectancy of humans varies depending on what country you live in.

In this project, we will investigate a dataset containing information about the average life expectancy in 158 different countries. We will specifically look at how a country's economic success might impact the life expectancy in that area.

Tasks

3/13 complete
Mark the tasks as complete by checking them off

Access the Data

1.
We've imported a dataset containing the life expectancy in different countries. The data can be found in the variable named `data`.
To begin, let's get a sense of what this data looks like. Inspect the `head()` to see the first 6 rows of the dataset.
Look at the names of the columns. What other pieces of information does this dataset contain?
You may want to comment out this call to `head()` after looking at the data.
The three column names are `"country"`, `"life_expectancy"`, and `"GDP"`. We've included information about the wealth of each country in addition to the life expectancy. We'll use this information later!

2.
Let's isolate the column that contains the life expectancy and store it in a variable named `life_expectancy`. To normally get a single column from a data frame using dplyr, you would use this syntax:

```
single_column <- data_frame_name %>%
  select(column_name)
```

Copy to Clipboard

Many of R's statistical functions, however, require a numeric vector, not a data frame column. In order to utilize the function for calculating quantiles, we need to convert the data frame column into a vector. We can do this by replacing the call to `select()` with the dplyr function `pull()`.

```
single_column <- data_frame_name %>%
  pull(column_name)
```

Copy to Clipboard

Make sure to pay attention to capitalization when using the column name!

Find the Quantiles

3.
We can now use R's statistical functions on that column! Let's use the `quantile()` function to find the quartiles of `life_expectancy`. Store the result in a variable named `life_expectancy_quartiles` and view the results.
Your call to the quantile function should look like this:

```
life_expectancy_quartiles <- quantile(____, c(0.25, 0.5, 0.75))
```

Copy to Clipboard

Fill in the name of the dataset you want to find the quantiles of. For us, this is the vector of values from the life expectancy column.

4.
Nice work! By looking at those three values you can get a sense of the spread of the data. For example, it seems like some of the data is fairly close together — a quarter of the data is between `72.5` years and `75.4` years.

Could you predict what the histogram might look like from those three number? Plot the histogram by using the following line of code. Does it look how you expected?

```
hist(life_expectancy)
```

Copy to Clipboard

5.

Let's take a moment to think about the meaning of these quartiles. If your country has a life expectancy of `70` years, does that fall in the first, second, third, or final quarter of the data? Click on the hint to see the answer!

`70` is between the first and second quartile, so it falls in the second quarter of the data.

Splitting the Data by GDP

6.

GDP is a measure of a country's wealth. Let's now use the GDP data to see if life expectancy is affected by this value.

Let's split the data into two groups based on GDP. If we find the median GDP, we can create two datasets for "low GDP countries" and "high GDP countries.

To start, let's isolate the `GDP` column and store it in a variable named `gdp`. This should be similar to how you isolated the life expectancy column.

7.

We now want to find the median GDP. You can use R's `median()` function, but since the median is also a quantile, we can call `quantile()` using `0.5` as the second argument.

Store the median in a variable named `median_gdp`. View that variable to see the median.

8.

Let's now grab all of the rows from our original dataset that have a GDP less than or equal to the median. The following code will do that for you:

```
low_gdp <- data %>%
  filter(GDP <= median_gdp)
```

Copy to Clipboard

Do the same for all of the rows that have a GDP higher than the median. Store those rows in a variable named `high_gdp`.

The line of code should look almost identical to the one above, but you should change the `<=` to `>`.

9.

Now that we've split the data based on the `GDP`, let's grab the values from the `life_expectancy` column to more easily analyze the data. The code to grab the `life_expectancy` values from the low gdp countries is given below:

```
low_gdp <- data %>%
  filter(GDP <= median_gdp) %>%
  pull(life_expectancy)
```

Copy to Clipboard

Do the same for the high gdp countries, using `pull()` to grab the `life_expectancy` values.

10.

Let's see how the life expectancy of each group compares to each other.

Find the quartiles of `low_gdp`. Store the quartiles in a variable named `low_gdp_quartiles`. View the results.

11.

Find the quartiles of the high GDP countries and store them in a variable named `high_gdp_quartiles`. This should look very similar to the last line of code you wrote. View the results.

Histogram and Conclusions

12.

By looking at the quantiles, you should get a sense of the spread and central tendency of these two datasets. But let's plot a histogram of each dataset to really compare them.

In the last code block, add these two lines of code:

```
hist(low_gdp,col='red')
hist(high_gdp,col='blue')
```

Copy to Clipboard

13.

We can now truly see the impact GDP has on life expectancy.

Once again, consider a country that has a life expectancy of `70` years. If that country is in the top half of GDP countries, is it in the first, second, third, or fourth quarter of the data with respect to life expectancy? What if the country is in the bottom half of GDP countries? Check the hint to see our thoughts.

Analyze Data with R

Blood Transfusion Analysis

https://www.codecademy.com/paths/analyze-data-with-r/tracks/statistics-in-r-skill-path/modules/hypothesis-testing-r/projects/hypothesis-testing-r-blood-transfusion

In this project, you will leverage your knoweldge of hypothesis testing to help drive informed decisions at a company inspecting blood transfusion data, Familiar.

You will use your combined knowledge of R and hypothesis testing to implement an analysis. The project will require implementation of standard deviation, one and two sample t-tests, as well as an understanding of the significance on the resulting p-value to give a recommendation on the results of the study.

Tasks

0/8 complete

Mark the tasks as complete by checking them off

One Sample T-Test

1.

Familiar want to know if their most basic package, the Vein Pack, actually has a significant impact on the subscribers. It would be a marketing goldmine if it could show that subscribers to the Vein Pack live longer than the general public.

Lifespans of Vein Pack users are given in the variable `vein_lifespans`, which has been loaded into `notebook.Rmd`. View `vein_lifespans`.

2.

You'd like to find out if the average lifespan of a Vein Pack subscriber is *significantly different* from the average life expectancy of `71` years.

Begin by finding mean lifespan of users of the Vein Pack, and save the result to `vein_lifespans_mean`. View `vein_lifespans_mean`.

3.

Now find the standard deviation of the lifespan of users of the Vein Pack. Save the result to `vein_lifespans_sd`, and view it.

4.

Use a One Sample T-Test to compare `vein_lifespans` to the average life expectancy, `71`. Save the result into a variable called `vein_pack_test`, and view it.

Are the results significant? Check the p-value of `vein_pack_test`. If it's less than `0.05`, the standard threshold, you've got significance!

Two Sample T-Test

5.

In order to differentiate Familiar's product lines, they would like you to compare the lifespan data for the Vein Pack with their more premium product, the Artery Pack.

Lifespans of Artery Pack users are given in the variable `artery_lifespans`, which has been loaded into `notebook.Rmd`. View `artery_lifespans`.

6.

Before you run a Two Sample T-test to compare the Vein Pack and the Artery pack, you want to learn more about the data collected on users of the Artery pack.

Begin by finding mean lifespan of users of the Artery Pack, and save the result to `artery_lifespans_mean`. View `artery_lifespans_mean`.

7.

Now find the standard deviation of the lifespan of users of the Artery Pack. Save the result to `artery_lifespans_sd`, and view it.

8.

Use a Two Sample T-Test to compare `vein_lifespans` to `artery_lifespans`. Save the result into a variable called `package_comparison_results`, and view it.

Are the results significant? Check the p-value of `package_comparison_results`. If it's less than `0.05`, the standard threshold, you've got significance!

````
---
title: "Blood Transfusion Analysis"
output: html_notebook
---

```{r error=TRUE}
# load data
load("vein_lifespans.Rda")
load("artery_lifespans.Rda")
```

```{r error=TRUE}
# view vein_lifespans here:

```

```{r error=TRUE}
# calculate vein_lifespans_mean here:


```

```{r error=TRUE}
````
````

```
# calculate vein_lifespans_sd here:

```

```{r error=TRUE}
# perform one sample t-test here:

```

```{r error=TRUE}
# view artery_lifespans here:
```

```{r error=TRUE}
# calculate artery_lifespans_mean here:

```

```{r error=TRUE}
# calculate artery_lifespans_sd here:

```

```{r error=TRUE}
# perform two sample t-test here:

```

Analyze Data with R

Explore the 1985 Cars Dataset

Have you ever wanted to have a collection of something? It could be trading cards, board games, art pieces, or whatever you want them to be. In this project, we'll suppose you collect something with a lot of data - cars!

Your car collection is almost complete - it's just missing one thing. You think about it for a while, then it comes to you - you don't have a car from 1985! But there are so many cars from 1985 available for you to collect! How will you know which 1985 car you will buy for your collection?

Luckily for you, we have a dataset that stores precisely that information. This dataset comes from the UCI Machine Learning Repository and is linked here. This dataset has been adapted so that the variable names are along the first rows.

As you'll notice in this project, though, this dataset isn't ready for analysis yet. You will have to make some changes along the way to clean and tidy up the dataset. Do that, and you'll have the ability to perform a great analysis!

Click the "Start" button below to get started!

Tasks

0/15 complete
Mark the tasks as complete by checking them off

Loading and Inspecting the Data

1.
What good is an analysis if we don't even have the tools to perform the said analysis? Some of the tools you will need for this analysis are the `readr` and `dplyr` tidyverse packages. Load the libraries at the top of the **notebook.Rmd** file so you can access the functions you will need later on.
2.
The last tool we need is the data itself! The file **cars85.csv** stores the data that comes from the UCI Machine Learning Repository. Load the file into a dataframe called `cars` to get started.
3.
It's always a good idea to inspect the data you load into R. It helps you to know what you are working with. Inspect `cars` with `head()` and `summary()`.
What kind of information do you have? What can you do with this information?
Each row in this dataframe is a single car, and each column stores some characteristic about that car. You want to get the best value for your collection, so you want to analyze as much as you can before buying. Doing so will help you make your choice easier!

Clean the Data

4.
After inspecting the dataframe, you notice something odd about the `normalized_losses` column. This column has a lot of entries that are question marks (`?`). This variable is not worth looking at since we don't have all the cars' expected losses.
Let's remove this column from the dataset. Select all columns from `cars` but `normalized_losses`. Save your new dataframe to `cars`.
5.
Print the column names of `cars`. Are they clear and descriptive?
6.
You know, `symboling` doesn't say anything to you at first glance. According to the UCI webpage, the `symboling` variable represents the car's risk factor. That variable name doesn't seem to go with the description. You should simplify this variable name to have it make more sense.
Update that column name in `cars` as follows:

`symboling` -> `risk_factor`

Print the column names of `cars` to confirm the names of the columns have changed.

Add a Column

7.

Your car collection means a lot to you. You want each car to be of value to you. What better way to do that than to buy a car with a lot of miles-per-gallon on the highways? To determine this, first, suppose only cars exceeding 30 mpg on the highways interest you. You seek to measure how different each car's highway mpg is from your 30 mpg threshold. Create a variable called `mpg_threshold` with the value `30`.

8.

Add a new column to `cars` called `mpg_diff_from_threshold`. This will measure how far each car's highway mpg is from 30 mpg. View the updated `cars` dataframe.

Filter and Arrange Rows

9.

You'll add a car to your collection only if it gets more than 30 miles per gallon on the highways. Filter the rows of `cars` to find all the cars where `mpg_diff_from_threshold` is greater than `0`. Save this new dataframe to `mpg_exceeds_threshold` and view it.

10.

Which cars have the highest miles per gallon on the highways? To find this, arrange the rows of `mpg_exceeds_threshold` by `mpg_diff_from_threshold` descending. Save this new dataframe as `mpg_exceeds_threshold`.

11.

Now suppose you want your next car to have a large engine. Order the rows of `cars` by `engine_size` descending. Save the new data frame to `ordered_by_engine_size`. View `ordered_by_engine_size`.

Specifying the Make of the Car

12.

There's a lot of makes of cars to choose from, but you may prefer one over the others. Which make do you prefer the most? Create a variable called `chosen_make` that contains the make you want to check. The hint below provides the list of makes to choose from.

13.

Filter `cars` to only include rows where the `make` column is equal to `chosen_make`. Save the new dataframe to `chosen_make_details`.

14.

Order the rows of `chosen_make_details` by `engine_size` descending and save the new dataframe to `chosen_make_details`. View `chosen_make_details`.

How large are the engines in each of the cars from that make that you chose? You can change the make stored in `chosen_make` to check out the engine sizes for other makes.

15.

The process of buying a new car can cause a lot of stress - you don't want to buy a car you won't like! You've now seen how performing an analysis can ease some of the stress of making the decision of which car to buy. You also get to add a nice new car to your collection! Great work!

Analyze Data with R

Cleaning US Census Data

https://www.codecademy.com/paths/analyze-data-with-r/tracks/working-with-data-in-r-skill-path/modules/learn-r-data-cleaning/projects/r-data-cleaning-us-census

You just got hired as a Data Analyst at the Census Bureau, which collects census data and finds interesting insights from it.

The person who previously had your job left you all the data they had for the most recent census. The data is spread across multiple `csv` files. They didn't use R, and they would manually look through these `csv` files whenever they wanted to find something. Sometimes they would copy and paste certain numbers into Excel for analysis.

The thought of it makes you shiver. This is not scalable or repeatable.

Your boss wants you to dig into the data and find some insights by the end of the day. Can you get this data into R and into reasonable shape so that you can perform your analysis?

If you get stuck or want to see an experienced programmer tackle this project, click "get help" to see a walkthrough video.

Tasks

1/17 complete
Mark the tasks as complete by checking them off

Load and Inspect the Data!

1.
Open some of the census `csv` files in the navigator. How are they named? What kind of information do they hold? What tools will you need to import and clean this data?
2.
In the first code block of `notebook.Rmd`, import the readr, dplyr, and tidyr libraries to aid you in your data cleaning.
3.
It will be easier to inspect the data stored in these files once you have it in a data frame. You can't even call `head()` on these `csv`s! How are you supposed to read them?
Begin by creating a variable called `files` and set it equal to the `list.files()` of all of the `csv` files you want to import.
4.
Read each file in `files` into a data frame using `lapply()` and save the result to `df_list`.
5.
Concatenate all of the data frames in `df_list` into one data frame called `us_census`.
6.
Inspect the `us_census` data frame by printing the column names, looking at the data types with `str()`, and viewing the `head()`.
What columns have symbols that will prevent calculations? What are the data types of the columns? Do any columns contain multiple kinds of information?

Remove and Reformat the Columns

7.
When inspecting `us_census` you notice a column `X1` that stores meaningless information. Drop the `X1` column from `us_census`, and save the resulting data frame to `us_census`. View the head of `us_census`.
8.

You notice that there are `6` columns representing the population percentage for different races. The columns include the percent symbol `%`. Remove the percent symbol `%` from each of the race columns (`Hispanic`,`White`,`Black`,`Native`,`Asian`,`Pacific`). Save the resulting data frame to `us_census`, and view the head.

9.

The `Income` column also incudes a `$` symbol along with the number representing median income for a state. Remove the `$` from the `Income` column. Save the resulting data frame to `us_census`. View the head of `us_census`.

10.

The `GenderPop` column appears to hold the male and female population counts. Separate this column at the `_` character to create two new columns: `male_pop` and `female_pop`. Save the resulting data frame to `us_census`, and view the head.

11.

You notice the new `male_pop` and `female_pop` columns contain extra characters `M` and `F`, respectively. Remove these extra characters from the columns. Save the resulting data frame to `us_census`, and view the head.

Update the Data Types

12.

Now that you have removed extra symbols from many of the columns that contain numerical data, you notice that the data type for these columns is still `chr`, or character. Convert all of these columns (`Hispanic`,`White`,`Black`,`Native`,`Asian`,`Pacific`,`Income`,`male_pop`,`female_pop`) to have a data type of numeric. Save the resulting data frame to `us_census`, and view the head.

13.

Take a second to look back at the `Hispanic`, `White`, `Black`, `Native`, `Asian`, and `Pacific` columns. The columns represent the population percentage for each race. Earlier you removed the `%` symbol, and then you just converted the column to numeric type. To make calculations easier, the column should now represent percentages in decimal form, where `50%` is equivalent to `0.50`. Update the values of these columns to be in decimal form, and save the resulting data frame to `us_census`. View the head of `us_census`.

Remove Duplicate Rows

14.

It's always a good idea to check if there are duplicate rows of data in a data set. Pipe `us_census` into the `duplicated()` function to see which rows are duplicated. Then pipe the result into `table()` to get a count of the duplicated rows.

15.

Since there are duplicates, update the value of `us_census` to be the `us_census` data frame with only unique/distinct rows.

16.

Confirm that there are no more duplicated rows in `us_census`. Pipe `us_census` into the `duplicated()` function to see which rows are duplicated. Then pipe the result into `table()` to get a count of the duplicated rows. You should expect to see no `TRUE`s!

17.

View the `head()` of `us_census`. The data frame is all clean and ready for analysis! What do you want to find out?

Analyze Data with R

A/B Testing for ShoeFly.com

https://www.codecademy.com/paths/analyze-data-with-r/tracks/working-with-data-in-r-skill-path/modules/learn-r-aggregates/projects/r-aggregates-shoefly

Our favorite online shoe store, ShoeFly.com is performing an A/B Test. They have two different versions of an ad, which they have placed in emails, as well as in banner ads on Facebook, Twitter, and Google. They want to know how the two ads are performing on each of the different platforms on each day of the week. Help them analyze the data using aggregate measures.

Tasks

0/11 complete
Mark the tasks as complete by checking them off

Analyzing Ad Souces

1.
Inspect the first few rows of `ad_clicks` using `head()`. What variables are stored in the columns of the data frame?
Click the hint to see a description of the columns of `ad_clicks`.

2.
Your manager wants to know which ad platform is getting the most views.
How many views (i.e., rows of the data frame) came from each `utm_source`? Group `ad_clicks` by `utm_source` and count the number of rows in each group. Save your result to `views_by_utm`, and view it.

3.
We want to know the percentage of people who clicked on ads from each `utm_source`. Let's start by finding the number of clicks per `utm_source`.
Group `ad_clicks` by `utm_source` and `ad_clicked` and count the number of rows in each group, naming the resulting column `count`. Save your answer to the variable `clicks_by_utm`, and view it.

4.
To find the percentage of people who clicked on ads from each `utm_source`, we need to add a new column to `ad_clicks` that stores the `count` of clicks (or the `count` of not clicking) divided by the total number of ad views.
Group `clicks_by_utm` by `utm_source` and pipe the result into `mutate()`, creating a new column `percentage` that is defined as `count/sum(count)`.
Save your result to `percentage_by_utm` and view it. Open the hint for a description of the `percentage_by_utm` data frame.

5.
Filter `percentage_by_utm` to remove all rows that do not describe clicked ads, and view it.
Was there a difference in click rates for each source?

Analyzing an A/B Test

6.
The column `experimental_group` tells us whether the user was shown ad A or ad B.
Were approximately the same number of people shown both ads? Group `ad_clicks` by `experimental_group` and count the rows, saving your result to `experiment_split`.
View `experiment_split` to see how users were split across the experiment groups!

7.
Using `group_by()` and the columns `ad_clicked` and `experimental_group`, check to see if a greater number of users clicked on ad A or ad B. Save your result to `clicks_by_experiment`, and view it.
Reveal the hint to see which ad was more effective.

8.
The Product Manager for the A/B test thinks that the clicks might have changed by day of the week.
Start by creating two data frames: `a_clicks` and `b_clicks`, which contain only the results for `A` group and `B` group, respectively.

9.
For each data frame (`a_clicks` and `b_clicks`), we want to find the number of users who clicked on the ad (and did not click on the ad) by `day`. Save the results to `a_clicks_by_day` and `b_clicks_by_day`, and view them.

10.
To find the percentage of people who clicked on the ads from each `day`, we need to add a new column to `a_clicks_by_day` and `b_clicks_by_day` that stores the `count` of clicks (or the `count` of not clicking) divided by the total number of ad views.
Group `a_clicks_by_day` and `b_clicks_by_day` by `day` and pipe the result into `mutate()`, creating a new column `percentage` that is defined as `count/sum(count)`.
Save your result to `a_percentage_by_day` and `b_percentage_by_day` and view them. Open the hint for a description of these data frames.

11.

Filter `a_percentage_by_day` and `b_percentage_by_day` to remove all rows that do not describe clicked ads, and view them.
Was there a difference in click rates for each day between the two ads? What happened over the course of the week?
Do you recommend that ShoeFly.com use ad A or ad B?
Reveal the hint for our analysis.

Analyze Data with R

Page Visits Funnel

Cool T-Shirts Inc. has asked you to analyze data on visits to their website. Your job is to build a funnel, which is a description of how many people continue to the next step of a multi-step process.

In this case, our funnel is going to describe the following process:

A user visits CoolTShirts.com
A user adds a t-shirt to their cart
A user clicks “checkout”
A user actually purchases a t-shirt

Tasks

0/12 complete
Mark the tasks as complete by checking them off
Funnel for Cool T-Shirts Inc.

1.
Inspect the data frames using `head()`:
`visits` lists all of the users who have visited the website
`cart` lists all of the users who have added a t-shirt to their cart
`checkout` lists all of the users who have started the checkout
`purchase` lists all of the users who have purchased a t-shirt

2.
Combine `visits` and `cart` using a *left join*.

3.
How long is the `visits` data frame?

4.
How many of the timestamps are `NA` for the column `cart_time`?
What do these null rows mean?

5.
What percent of users who visited Cool T-Shirts Inc. ended up *not* placing a t-shirt in their cart?

6.
Repeat the left join for `cart` and `checkout` and count `NA` values. What percentage of users put items in their cart, but did not proceed to checkout?

7.
Join all four steps of the funnel, in order, using a series of *left joins*. Save the results to the variable `all_data`.
Examine the result using `head()`.

8.
What percentage of users proceeded to checkout, but did not purchase a t-shirt?

9.
Which step of the funnel is weakest (i.e., has the highest percentage of users not completing it)?
How might Cool T-Shirts Inc. change their website to fix this problem?

Average Time to Purchase

10.
Using the giant joined data `all_data` that you created, let’s calculate the average time from initial visit to final purchase. Start by adding the following column to your DataFrame:

```
all_data <- all_data %>%
  mutate(time_to_purchase = purchase_time - visit_time)
```

Copy to Clipboard

11.

Examine the results using:

```
head(all_data)
```

Copy to Clipboard

12.

Calculate the average time to purchase using the following code:

```
time_to_purchase <- all_data %>%
  summarize(mean_time = mean(time_to_purchase, na.rm = TRUE))
time_to_purchase
```

Analyze Data with R

Visualizing Carbon Dioxide Levels

Climate scientists have measured the concentration of carbon dioxide (CO2) in the Earth's atmosphere dating back thousands of years. In this project, we will investigate two datasets containing information about the carbon dioxide levels in the atmosphere. We will specifically look at the increase in carbon dioxide levels over the past hundred years relative to the variability in levels of CO2 in the atmosphere over eight millennia.

These data are calculated by analyzing ice cores. Over time, gas gets trapped in the ice of Antarctica. Scientists can take samples of that ice and see how much carbon is in it. The deeper you go into the ice, the farther back in time you can analyze!

The first dataset comes from World Data Center for Paleoclimatology, Boulder and NOAA Paleoclimatology Program and describes the carbon dioxide levels back thousands of years "Before Present" (BP) or before January 1, 1950.

The second dataset explores carbon dioxide starting at year zero up until the recent year of 2014. This dataset was compiled by the Institute for Atmospheric and Climate Science (IAC) at Eidgenössische Technische Hochschule in Zürich, Switzerland.

In order to understand the information in these datasets, it's important to understand two key facts about the data:

> The metric for carbon dioxide level is measured as parts per million or CO2 ppmv. This number describes the number of carbon dioxide molecules per one million gas molecules in our atmosphere.
> The second metric describes years before present, which is "a time scale used mainly in … scientific disciplines to specify when events occurred in the past… standard practice is to use 1 January 1950 as the commencement date of the age scale, reflecting the origin of practical radiocarbon dating in the 1950s. The abbreviation "BP" has alternatively been interpreted as "Before Physics" that is, before nuclear weapons testing artificially altered the proportion of the carbon isotopes in the atmosphere, making dating after that time likely to be unreliable." This means that saying "the year 20 BP" would be the equivalent of saying "The year 1930."

Let's get started!

Tasks

18/18 complete
Mark the tasks as complete by checking them off
Carbon Dioxide over Time Line Graph

1.
Let's first explore the dataset containing the data from the World Data Center for Paleoclimatology, Boulder and NOAA Paleoclimatology Program. Import the `"carbon_dioxide_levels.csv"` and save it to a new variable named `noaa_data`.

```
read_csv("filename.csv")
```

Copy to Clipboard

2.
Inspect the head of the data frame. What are the names of the two columns? What types of values are in each?

3.
Let's visualize this data. First, create a new variable named `noaa_viz` that is equal to a new `ggplot()` object and assign `noaa_data` as its `data` argument. Be sure to state the name of the variable after you define it so that it is rendered to the R notebook.

4.
Define your scales by creating an aesthetic mapping that maps `Age_yrBP` on the x-axis and `CO2_ppmv` on the y-axis as part of the canvas.

5.
In climate science, it's common to create line graphs to best portray the fluctuations in the levels of carbon dioxide. Add a `geom_line()` layer to the `noaa_viz` plot.
6.
Let's add context to the plot and improve its legibility. Title the plot `"Carbon Dioxide Levels From 8,000 to 136 Years BP"` and add a subtitle that cites the data `"From World Data Center for Paleoclimatology and NOAA Paleoclimatology Program"`.
7.
Tweak the axis labels so they are more descriptive than the column headers. The x-axis should read `"Years Before Today (0=1950)"` and the y-axis should read `"Carbon Dioxide Level (Parts Per Million)"`
8.
Currently, the order of the years is counterintuitive. Since the most recent date is the date closest to 0, or 1950 as Before Physics is described, we want the years on the x-axis arranged in descending order. Add this to `noaa_viz`:

```
noaa_viz + scale_x_reverse(lim=c(800000,0))
```

Copy to Clipboard

Carbon Dioxide Levels in the last Two Millennia

9.
In the second code block, let's explore the second dataset containing the data for the last 2014 years. Import the `"yearly_co2.csv"` file and save it to a new variable named `iac_data`.
10.
Inspect the head. What are the names of the four columns? What types of values are in each? Note that the `data_mean_global` is an equivalent metric to `CO2_ppmv`. We will not be using the other two columns in this project. What's different about the year column in this dataset?
11.
Again, let's create a new `ggplot()` object named `iac_viz` and associate `iac_data` as its data argument. Let's make a new variable named `iac_viz`. Be sure to state the name of the variable after you define it so that it is rendered to the R notebook.
12.
Define your scales by creating an aesthetic mapping that maps `year` on the x-axis and `data_mean_global` on the y-axis as part of the canvas.
Note: The dataset column headers are different than the ones in the previous data frame. Years are chornological starting from 0 up to 2014, not in terms of BP. The `data_mean_global` references the same metric as `CO2_ppmv` for carbon dioxide average parts per million in the earth's atmosphere.
13.
A line graph also makes sense for these data. Let's explore how much carbon dioxide was stored in the atmosphere over the past two millennia by adding a `geom_line()` layer to the `iac_viz` plot.
14.
This plot still needs labels to add context to the plot. Title the plot `"Carbon Dioxide Levels over Time"` and add a subtitle that cites the data `"From Institute for Atmospheric and Climate Science (IAC)."`
15.
Tweak the axis labels so they are more descriptive than the column headers. The x-axis should read `"Year"` and the y-axis should read `"Carbon Dioxide Level (Parts Per Million)"`
16.
Let's highlight the rise in carbon dioxide levels by adding a horizontal line that represents the maximum level in the first chart spanning over 8,000 years of carbon dioxide data. On a new line of code in the block, create a new variable named `millennia_max` and retrieve the maximum value of the `CO2_ppmv` column in the `noaa_data`. Print the value so you can see what it is.
17.
Now that we have the maximum number in the `noaa_data` let's map it on our `iac_data` plot. There's a geom in ggolot called `geom_hline()` that plots a horizontal line. Add a `geom_hline()` layer to `iac_viz` that has a `yintercept` value in its aesthetic mapping of `millennia_max`.

```
viz + geom_hline(aes(yintercept=somevalue))
```

```
Copy to Clipboard
```

18.

Add one more argument to the horizontal line's aesthetic mapping so that the legend can display information about what the line represents. Assign the value of the `linetype` argument as `"Historical CO2 Peak before 1950"`

What do you notice has happened in the last 100 years relative to the last 8 millennia?

```
viz + geom_hline(aes(yintercept=somevalue, linetype="Some string"))
```

Analyze Data with R

Museums and Nature Centers

https://www.codecademy.com/paths/analyze-data-with-r/tracks/data-visualization-in-r-skill-path/modules/intermediate-data-visualization-with-ggplot-2/projects/data-visualization-in-r-museums

There are thousands of museums, aquariums, and zoos across the United States. In this project, we'll take a look at the distribution of these institutions by geographic region, type, and revenue.

Our data is compiled from administrative records from the Institute of Museum and Library Services, IRS records, and private foundation grantmaking records. This data reflects the status of each institution as of 2013. For each institution, we have information on its name, type, and location. Each institution also has a parent organization – for example, if a museum housed at a university, its parent organization is the university where it resides. Financial data on annual revenue is available at the parent organization level.

We'll be creating several different data visualizations in this project. Some of these may feel challenging, but they'll all relate back to the concepts we've learned in this lesson. If you get stuck on one plot, feel free to move on to a different section. We've provided hints throughout, and you can consult the lesson materials as well.

Tasks

0/19 complete
Mark the tasks as complete by checking them off

Data Exploration

1.
Let's start by loading our dataset. We've provided a file named `museums.csv`. Load this file into a data frame named `museums_df`.

2.
Take a look at the head of this data frame. Make sure to click through using the arrows in the header to see all the available columns.
The `Museum.Name` column represents the name of each individual institution, while the `Legal.Name` column represents the name of each institution's parent entity. For example, if `"Codecademy University"` has two museums on campus, each of those museums would have their own names under `Museum.Name` and both would share the same `Legal.Name` i.e. `"Codecademy University"`.

Explore Institutions by Type

3.
In this section, we'll explore the distribution of institutions in our dataset by type. Our data frame contains a column called `Museum.Type` describing what kind of museum each location is – a history museum, a zoo, an aquarium, etc. Create and print a bar plot called `museum_type` that maps `Museum.Type` to the `x` axis and counts the frequency of each type on the `y` axis. Which category is most common?

4.
The plot we just created is hard to read because our categories are long. Add a `scale_x_discrete()` layer to customize our `x` axis, using the function `scales::wrap_format(8)` to reformat our labels. `wrap_format()` is a function from the `scales` packages which comes included with `ggplot2`. By setting the value of `wrap_format()` to `8`, we are telling it that the maximum width per line should be no more than `8` characters.
Now we should be able to see which category is most common. Great job! Give yourself a pat on the back.

5.
We've included a boolean (`TRUE` or `FALSE`) column in our data frame called `Is.Museum`. The `TRUE` category includes typical museums like art, history, and science museums. The `FALSE` category includes zoos, aquariums, nature preserves, and historic sites, which are included in this data but aren't what most people think of when they hear the word "museum."

Create a new bar plot called `museum_class`, mapping `Is.Museum` to the `x` axis. Since "TRUE" and "FALSE" aren't very descriptive, use `scale_x_discrete()` to rename the `x` axis labels to more easily understood terms – for example, "Museum" vs "Non-Museum".

6.

Instead of looking at the distribution across the entire United States, maybe we're just interested in a few states. Filter `museums_df` to include a few states you might be interested in, using the `State..Administrative.Location.` column; for example, we can choose `IL`, `CA`, and `NY`. Call this filtered data frame `museums_states`.

After creating `museums_states`, recreate our bar plot showing the distribution of museums vs non-museums and use `facet_grid()` to display each state's distribution in a separate panel. Call this plot `museum_facet`. How does the distribution of museum vs non-museum vary across the states you chose?

7.

Our data also contains information on each museum's region, representing groups of states. Create a stacked bar plot using `museums_df` showing the count of museums by region (`Region.Code..AAM.`), mapping `Is.Museum` to the `fill` aesthetic. Convert `Region.Code..AAM.` to a factor (e.g. `factor(Region.Code..AAM.)`) so `ggplot2` plots its levels as discrete rather than continuous values. Call this plot `museum_stacked`.

8.

Our plot is hard to read – right now, we don't know what the region numbers correspond to. Use `scale_x_discrete()` to rename the numeric labels to text according to the following table.

	Code	Region
1		New England
2		Mid-Atlantic
3		Southeastern
4		Midwest
5		Mountain Plains
6		Western

Similarly, add a `scale_fill_discrete()` layer to relabel the "TRUE" and "FALSE" labels in our legend to "Museum" and "Non-Museum".

Based on the plot we created, which region has the most museums?

9.

Rather than seeing counts, perhaps we're more interested in the percentage of museums vs non-museums by region. Transform the plot we just created to a stacked bar plot showing values out of 100% by passing `position = "fill"` to our `geom_bar()` layer. Apply the `scales::percent_format()` function to transform our `y` axis labels into percentage values.

How does the distribution of museum types vary by region?

10.

Our graph looks pretty good! However, our axes titles are a little non-descript. Using the `labs()` layer, let's title this plot "Museum Types by Region", relabel the `x` axis title as "Region", relabel the `y` axis title as "Percentage of Total", and relabel the `fill` legend title as "Type".

Now, someone can take a look at this plot and immediately understand what is being described. There were a lot of steps here, but now our plot is clear and professional. Give yourself a pat on the back, and feel free to take a 10 minute coffee break before the next section!

Explore Institutions by Revenue

11.

For the next few tasks, we'll switch to looking at how much money each institution brought in and how that varies across geographies. Because we only have revenue data at the parent organization level, we'll want to first filter our dataset to omit any duplicates. Next, we'll create a few data frames from our starting data to look at different groups of museums by how much money they bring in.

Create a new data frame called `museums_revenue_df` that retains only unique values of `Legal.Name` in `museums_df`. Additionally, filter this data frame to include only entities with `Annual.Revenue` greater than 0.

Create a second data frame from `museums_revenue_df` (the first data frame we created in this task) called `museums_small_df` that retains only museums with `Annual.Revenue` less than $1,000,000.

Create a third data frame from `museums_revenue_df` (the first data frame we created in this task) called `museums_large_df` that retains only museums with `Annual.Revenue` greater than $1,000,000,000.

12.

Let's start by visualizing the distribution of annual revenue for our small museums dataset. Create a histogram called `revenue_histogram` using `museums_small_df` with `Annual.Revenue` mapped to the `x` axis. Experiment with different `binwidth` values to see what works best for our data, considering that our `x` axis variable ranges from 0 to $1,000,000.

13.

Our `x` axis is a little hard to read. Let's make it more clear! Add a `scale_x_continuous()` layer applying the function `scales::dollar_format()` to our `x` axis labels. `dollar_format()` is a function from the `scales` library included in `ggplot2` that adds dollar signs and commas to monetary data.

14.

Now, let's look at the variation in revenue for large museums by region. Create a boxplot called `revenue_boxplot` using `museums_large_df`, mapping `Region.Code..AAM.` to the `x` axis and `Annual.Revenue` to the `y` axis. Remember to convert `Region.Code..AAM.` to a factor (e.g. `factor(Region.Code..AAM.)`) so `ggplot2` plots its levels as discrete rather than continuous values. Use `scale_x_discrete()` to rename the numeric region codes to their text equivalents.

15.

The plot we just created is a little hard to read, since there's one outlier so far above all the other data points. This one museum made a lot of money in 2013! Let's zoom in so we can see the rest of our boxes more clearly. Add a `coord_cartesian()` layer setting `ylim` to `c(1e9, 3e10)`. This tells our plot to zoom in on the `y` axis range between $1,000,000,000 and $30,000,000,000. How do the median, 75th, and 25th quantiles vary by region?

16.

Though we can see the distribution by region more clearly now, our `y` axis label is hard to understand. Let's reformat our `y` axis as billions of dollars. The function defined below will convert values like $1,000,000,000 to $1B, which is much easier to read. Add a `scale_y_continuous()` layer to map our `y` axis labels using this function.

```
function(x) paste0("$", x/1e9, "B")
```

Copy to Clipboard

We've made our box plot much clearer to read. Great work! If you're feeling tired, you can always take a quick break – you've earned it!

17.

Now, let's take a look at revenue across all museums in our dataset. Using `museums_revenue_df`, create a bar plot called `revenue_barplot` mapping `Region.Code..AAM.` to the `x` axis and `Annual.Revenue` to the `y` axis. Remember to transform `Region.Code..AAM.` to a factor (`factor(Region.Code..AAM.)`) so it shows up as discrete values.

Use `stat = "summary"` and `fun = "mean"` to calculate and display the mean revenue by region. Apply the appropriate `x` and `y` axis label transformations to make our labels more clear. Which region has the highest and lowest mean revenues?

18.

Once again, use the `labs` layer to make our plot more clear. Title the plot "Mean Annual Revenue by Region", relabel the `y` axis title to "Mean Annual Revenue", and relabel the `x` axis title to "Region".

19.

Finally, let's add some error bars to our means. We'll need to calculate our standard errors before creating our plot. You can do so using the following code:

```
museums_error_df <- museums_revenue_df %>%
  group_by(Region.Code..AAM.) %>%
  summarize(
    Mean.Revenue = mean(Annual.Revenue),
    Mean.SE = std.error(Annual.Revenue)) %>%
  mutate(
    SE.Min = Mean.Revenue - Mean.SE,
    SE.Max = Mean.Revenue + Mean.SE)
```

Copy to Clipboard

Add error bars to our mean revenue by geography bar plot using the `geom_errorbar()` layer. Call this new plot `revenue_errorbar`. Our new `y` variable is our calculated `Mean.Revenue` column. Make sure to change the `stat` being used, since we're now displaying our calculated means as is rather than calculating them as we create the plot. Which regions have more or less variability around their mean revenues?

Congratulations – you've completed this project! Awesome work today.

````
---
title: "Museums and Nature Centers"
output:
  html_document:
    df_print: paged
---

```{r data, message=FALSE}

library(dplyr)
library(ggplot2)
library(stringr)
library(tidyr)
library(plotrix)

```

## Data Exploration

```{r load, message=FALSE}
# Load file as data frame
museums_df <- read.csv('museums.csv')
```

```{r inspect, message=FALSE}
# Inspect data frame
head(museums_df)
```

## Museums by Type

```{r barplot, message=FALSE}
# Create and print bar plot by type
museum_type <-
  ggplot(museums_df, aes(x = Museum.Type)) +
  geom_bar() +
  scale_x_discrete(
    labels = scales::wrap_format(8))
````
````

````
museum_type
```

```{r barplot_museum, message=FALSE}
# Create and print bar plot by museum vs non-museum
museum_class <-
  ggplot(museums_df, aes(x = Is.Museum)) +
  geom_bar() +
  scale_x_discrete(
    labels = c(
      "TRUE" = "Museum",
      "FALSE" = "Non-Museum"))

museum_class
```

```{r barplot_type, message=FALSE}
# Filter data frame to select states
museums_states <- museums_df %>%
  filter(
    grepl('IL|CA|NY',
    State..Administrative.Location.))

# Create and print bar plot with facets
museum_facet <-
  ggplot(museums_states, aes(x = Is.Museum)) +
  geom_bar() +
  scale_x_discrete(
    labels = c(
      "TRUE" = "Museum",
      "FALSE" = "Non-Museum")) +
  facet_grid(
    cols = vars(State..Administrative.Location.))

museum_facet
```

```{r barplot_stack, message=FALSE}
# Create and print stacked bar plot
museum_stacked <-
  ggplot(museums_df,
    aes(
      x = factor(Region.Code..AAM.),
      fill = Is.Museum)) +
  geom_bar(position = "fill") +
  scale_x_discrete(
    labels = c(
      "1"="New England",
      "2"="Mid-Atlantic",
      "3"="Southeastern",
      "4"="Midwest",
      "5"="Mountain Plains",
      "6"="Western")) +
  scale_y_continuous(
    labels = scales::percent_format()) +
  scale_fill_discrete(
    labels = c(
      "TRUE" = "Museum",
      "FALSE" = "Non-Museum")) +
  labs(
    title = "Museum Types by Region",
    x = "Region",
    y = "Percentage of Total",
    fill = "Type"
  )

museum_stacked
````

```

## Museums by Revenue

```{r process, message=FALSE}
# Filter data frame
museums_revenue_df <- museums_df %>%
  distinct(Legal.Name, .keep_all = TRUE) %>%
  filter(Annual.Revenue > 0)

# Filter for only small museums
museums_small_df <- museums_revenue_df %>%
  filter(Annual.Revenue < 1e6)

# Filter for only large museums
museums_large_df <- museums_revenue_df %>%
  filter(Annual.Revenue > 1e9)
```

```{r histogram, message=FALSE}
# Create and print histogram
revenue_histogram <-
  ggplot(museums_small_df,
    aes(x = Annual.Revenue)) +
  geom_histogram(binwidth = 1e4) +
  scale_x_continuous(
    labels = scales::dollar_format())

revenue_histogram
```

```{r boxplot, message=FALSE}
# Create and print boxplot
revenue_boxplot <-
  ggplot(museums_large_df,
    aes(
      x = factor(Region.Code..AAM.),
      y = Annual.Revenue)) +
  geom_boxplot() +
  coord_cartesian(ylim = c(1e9, 3e10)) +
  scale_y_continuous(
    labels = function(x) paste0("$", x/1e9, "B")) +
  scale_x_discrete(
    labels = c(
      "1"="New England",
      "2"="Mid-Atlantic",
      "3"="Southeastern",
      "4"="Midwest",
      "5"="Mountain Plains",
      "6"="Western"))

revenue_boxplot
```

```{r mean, message=FALSE}
# Create and print bar plot with means
revenue_barplot <-
  ggplot(museums_revenue_df,
    aes(
      x = factor(Region.Code..AAM.),
      y = Annual.Revenue)) +
  geom_bar(stat = "summary", fun = "mean") +
  scale_y_continuous(
    labels = scales::dollar_format()) +
  scale_x_discrete(
    labels = c(
      "1"="New England",
```
```

```
      "2"="Mid-Atlantic",
      "3"="Southeastern",
      "4"="Midwest",
      "5"="Mountain Plains",
      "6"="Western")) +
  labs(
    title = "Mean Annual Revenue by Region",
    y = "Mean Annual Revenue",
    x = "Region")

revenue_barplot
```

```{r mean_errorbar, message=FALSE}
# Calculate means and standard errors
museums_error_df <- museums_revenue_df %>%
  group_by(Region.Code..AAM.) %>%
  summarize(
    Mean.Revenue = mean(Annual.Revenue),
    Mean.SE = std.error(Annual.Revenue)) %>%
  mutate(
    SE.Min = Mean.Revenue - Mean.SE,
    SE.Max = Mean.Revenue + Mean.SE)

# Create and print bar plot with means and standard errors
revenue_errorbar <-
  ggplot(museums_error_df,
    aes(
      x = factor(Region.Code..AAM.),
      y = Mean.Revenue)) +
  geom_bar(stat = "identity") +
  geom_errorbar(
    aes(ymin = SE.Min, ymax=SE.Max), width=0.2) +
  scale_y_continuous(
    labels = scales::dollar_format()) +
  scale_x_discrete(
    labels = c(
      "1"="New England",
      "2"="Mid-Atlantic",
      "3"="Southeastern",
      "4"="Midwest",
      "5"="Mountain Plains",
      "6"="Western")) +
  labs(
    title = "Mean Annual Revenue by Region",
    y = "Mean Annual Revenue",
    x = "Region")

revenue_errorbar
```

Analyze Data with R

Predicting Income with Social Data

The Panel Survey of Income Data is the longest running longitudinal household survey in the world. Survey responses related to social, economic, and health outcomes have been collected from the families and their descendants since 1968. This dataset is widely used by social scientists to investigate the relationship between individual characteristics, like gender or age, and broader socioeconomic outcomes like education achievement and lifetime income. In this project, you'll have the chance to use PSID data and linear regression to predict the labor-derived income of survey respondents based on the following set of variables:

`gender`: the gender, female-identifying, male-identifying, or other, of a respondent
`age`: the age of the respondent
`married`: the marital status, unmarried, married, or divorced, of a respondent
`employed`: the employment status of the respondent at the time of survey collection
`educated_in_us`: whether the respondent went to primary school in the United Statues
`highest_degree`: the highest educational degree obtained by the respondent
`education_years`: the total number of years of formal education completed by the respondent
`labor_income`: the yearly income earned by the respondent from a salary or hourly employment

Unlike other data science methods, **linear regression models will allow us to not only predict the value of a respondent's income, but provide information on *how* a variable impacts respondent income.** Let's dive into the social patterns that underly Americans' incomes!

Tasks

0/22 complete
Mark the tasks as complete by checking them off
Clean and check data assumptions

1.
First, let's checkout the structure of our dataset and confirm it contains all the variables described in the introduction. Call `str()` on dataset, which has been saved to a variable called `psid`.
2.
Create a bar chart that plots the distribution of `age` in our `psid` dataset using `ggplot`'s `geom_bar`; do any of the observed values seem unrealistic?
3.
Considering we are interested in predicting the **labor income** of survey respondents, it would be reasonable to filter our data so that it only includes respondents of working age, roughly between 18 and 75. Use `dplyr`'s `filter()` method to exclude observations with `age < 18` or `age > 75`; save the result to a variable called `psid_clean`.
4.
Let's confirm that our call to `filter()` properly truncated our data. Create another bar chart of `psid`'s `age` column using `geom_bar`.
5.
Now let's perform the same check for the education level of respondents. Create a boxplot of the distribution of `education_years` using `geom_boxplot()`.
6.
Again, some of the values in our observed data do not make much sense– how could someone achieve 100 years of formal education? Let's use `filter()` to limit our dataset to observations where `education_years` is between `5` and `25`.
7.

Finally, let's check our outcome variable, `labor_income`. Create a boxplot of the distribution of `labor_income` using `geom_boxplot()`.

8.

Hmm, the scales on our `labor_income` boxplot are difficult to interpret; let's take a look at a quantitative representation of the variable distribution using by calling `summary()` on `labor_income`. What does this output tell us about the distribution of income in our dataset?

9.

Let's make sure we fully understand how this highly skewed income distribution relates to our key variables in our dataset. Create a scatterplot of average income by age using `dplyr`'s `group_by` and `summarise()` functions, along with `ggplot`'s `geom_point()`.

Which ages seem most likely to have zero `labor_income`?

Build model and assess fit

10.

Now we can build a model! Let's specify our training and test datasets.

- Change the dataset reference within `sample()` from `psid` to `psid_clean`.
- Using the `sample` variable already defined in the workspace, create a `train` data frame that includes all observations in `sample`, then create a `test` data frame that includes all observations **not** in `sample`.

11.

Build a simple linear model that regresses `education_years` on `labor_income`. Don't forget to use our `train` dataset!

12.

Let's compare our model against an LOESS smoother to see where our model predictions differ most substantially from the average observed value.

- Pass in `aes(education_years, labor_income)` into a call to `ggplot()`; don't forget to use our `train` dataset.
- Add a call to `geom_point()` to plot our observed values
- Add a call to `geom_smooth()`, passing in `method = "lm"` as a parameter.
- Add a call to `geom_smooth()` passing in `se = FALSE` and `color = "red"` as parameters.

How closely does our model align with the LOESS smoother?

13.

Using the R-squared metric, quantify the fit of `model`. Extract `r.squared` from our model output, multiply the result by `100`, and save the result to a variable called `r_sq`.

14.

Uncomment the following f-string to see how we would provide a narrative around `model`'s R-squared value.

Build comparison model and analyze results

15.

While our `model` was a good start, it seems reasonable to expect that other variables in our dataset, like `age` or `gender`, also have an impact on `labor_income`. Build a second model, `model_2`, which regresses `education_years`, `age`, and `gender` on `labor_income`. Don't forget to use our `train` dataset.

16.

Using the R-squared metric, quantify the fit of `model_2`. Extract `r.squared` from our model output, multiply the result by `100`, and save the result to a variable called `r_sq_2`.

17.

Uncomment the following f-string to see how we would provide a narrative around `model_2`'s R-squared value.

18.

Let's also provide a graphic representation of our `model_2` fit by plotting the observed and predicted values of `labor income using` add_predictions()`from the`modelr` package:

- Using our `test` data, add a call to `add_predictions()`, passing in `model_2`
- Add a call to `ggplot()`, passing `age` and `labor_income` as parameters to `aes()`
- Add a call to `geom_point()` to plot our observed values
- Add a call to `geom_line()`, explicitly setting in `pred` as a `y` value in `aes()`, and passing in `color = "blue"` as a parameter.

19.

With `model_2` as a substantial improvement over `model`(based on R-squared score), let's dive into results! Call `summary()` on `model_2` and take a first look at the coefficient values. Try and answer the following questions:

Do `education_years`, `age`, and `gender` all have a significant impact on `labor_income` ?
`gender` is a boolean categorical variable; how should we interpret its' coefficient value?
Which variable has the largest effect on `labor_income`?

20.

Extract the value of the `education_years` coefficent and assign the result to a variable called `education_coefficent`.

21.

Uncomment the following f-string to see how we would provide a narrative around `model_2`'s `education_years` coefficient.

22.

Great work! You've successfully cleaned and analyzed a real-world dataset, built and iterated towards a best-fit model, and provided an interpretation of your results!

Feel free to keep trying different combinations of predictor variables to see if you can improve upon `model_2`.

aa